

I think this is an excellent choice because it naturally showcases **LLMs, Agentic AI, distributed systems, cloud-native engineering, event-driven architecture, security, observability, and production operations** in a single project.

One suggestion before we begin:

Don't build a project. Build a platform.

This should look like something an enterprise would actually deploy internally.

Project Name

Some ideas:

- EnterpriseOS AI
- NexusAI
- Aegis Enterprise
- Synapse Enterprise
- CortexOps
- EnterpriseFlow AI
- OpenEnterprise AI

I personally like **CortexOps**.

"An Open Source AI Workforce Platform"

Vision

Imagine every employee in an organization has access to AI workers instead of AI chatbots.

Instead of saying

"Generate a deployment"

they simply assign work.

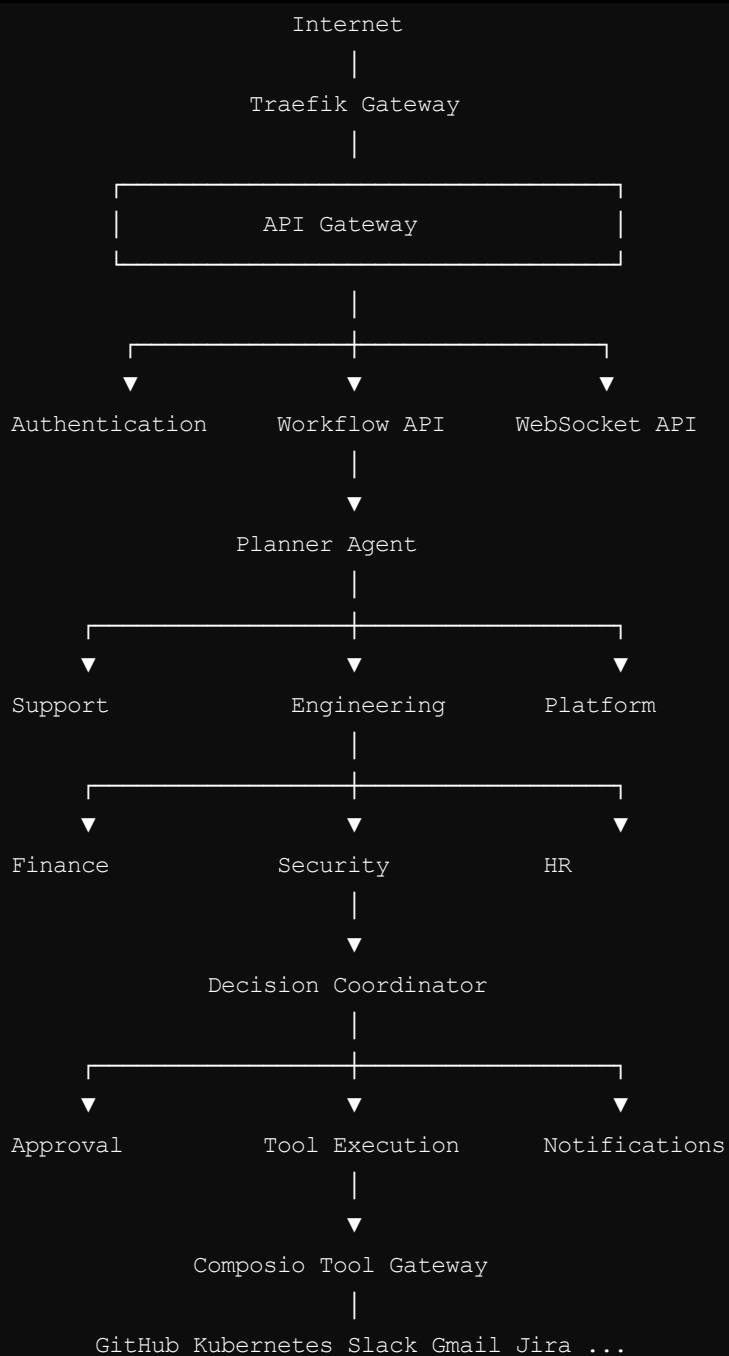
```
Deploy version 2.4 to staging after running regression tests.  
If everything passes,  
promote to production.  
Notify Slack.  
Create Jira deployment ticket.
```



The platform figures out

- planning
- approvals
- execution
- monitoring
- rollback
- reporting

High Level Architecture



Technology Stack

This is where I'd heavily favor mature open-source projects.

Backend

Component	Choice	Reason
Language	Python	Best AI ecosystem
API	FastAPI	Standard
Async	asyncio	Native
Validation	Pydantic	Excellent models
Dependency Injection	dishka or punq	Lightweight
ORM	SQLAlchemy 2	Mature
Database	PostgreSQL	Production standard

AI

Instead of LangChain.

Use

LangGraph



Why?

Because

- state machines
- workflows
- retries
- persistence
- human approvals

fit perfectly.

LLM Gateway

LiteLLM



Supports

- Ollama
- OpenAI
- Anthropic
- Gemini
- Groq
- OpenRouter

One API.

Local Models

Ollama



Examples

- Qwen3
 - Llama 3.3
 - DeepSeek
 - Mistral
 - Gemma
-

Embeddings

BGE



or

nomic-embed-text



Vector Database

Qdrant



Better than Chroma.

Production ready.

Memory

Hybrid memory.

Redis



Short-term

Qdrant



Semantic memory

Postgres



Long-term structured memory

Event Architecture

Avoid synchronous architecture.

Everything becomes an event.

TicketCreated



↓

Support Agent

↓

IncidentDetected

↓

Engineering Agent

↓

DeploymentRequested

↓

ApprovalRequested

↓

DeploymentFinished

↓

CustomerNotified

Message Broker

Use

NATS JetStream



instead of RabbitMQ.

Reasons

- lightweight
- Kubernetes native
- high throughput
- persistence
- request reply
- streaming

Workflow Engine

This is one of the biggest decisions.

Instead of Celery.

Use

Temporal



Because

- retries
- persistence
- resumability
- human approvals
- timers
- workflow history

This is exactly what enterprise AI needs.

Observability

Every service emits

Logs

Metrics

Traces

Logging

Loki



Metrics

Prometheus



Visualization

Grafana



Tracing

OpenTelemetry



↓

Jaeger

AI Observability

Phoenix



Track

- prompts
- tokens
- latency
- hallucinations
- traces

Kubernetes

Every microservice becomes

Deployment

Service

Ingress

ConfigMap

Secret

NetworkPolicy

ServiceAccount



Package everything with

Helm



Every service gets

values.yaml

deployment.yaml

service.yaml

ingress.yaml

hpa.yaml



configmap.yaml

secret.yaml

GitOps

ArgoCD



Git becomes source of truth.

Authentication

Use

Keycloak



Supports

OIDC

OAuth

JWT

RBAC

LDAP

Secret Management

Never store secrets.

Use

Hashicorp Vault



Agents

Instead of one agent

Create departments.

Planner

Responsibilities

- Understand request
 - Break tasks
 - Assign departments
-

Support

Tools

Slack

Email

Knowledge Base

Confluence

Engineering

Tools

GitHub

GitLab

Docker

Platform

Tools

Kubernetes

Helm

Terraform

ArgoCD

Finance

Stripe

Invoices

Refunds

Security

Vault

GitHub

Dependency Scanner

HR

Calendar

Google Workspace

Agent Architecture

Every agent should be identical internally.

Input

↓

Memory

↓

Planning

↓

Tool Selection

↓

Execution



↓

Reflection

↓

Output

This allows adding new departments without changing the framework.

Internal Agent Structure

Agent

↓

State

↓

Planner

↓

Memory

↓

Reasoner

↓

Tool Manager

↓

Validator

↓

Executor

↓

Reporter



Composio

Instead of calling APIs directly

Every integration goes through Composio.

Examples

GitHub

Slack

Jira

Linear

Gmail

Google Calendar

Discord

Notion

Confluence

Docker

Kubernetes

UI

I wouldn't build a chatbot.

I'd build an Operations Dashboard.

Pages

Dashboard

Workflows

Departments

Agents

Approvals



Tool Calls

Logs

Memory

Audit

Settings

Frontend

NextJS



React

Tailwind

shadcn/ui

TanStack Query

React Flow

Workflow Visualization

Use

React Flow



to show

Planner



↓

Engineering

↓

Deployment

↓

Slack

↓

Approval

↓

Production

live.

Storage

Postgres



Users

Departments

Tasks

Approvals

Workflows

Audit

Memory Metadata

Qdrant

Embeddings



Redis

Cache



Sessions

Temporary memory

MinIO

Documents



Logs

Artifacts

Attachments

Deployment

Development

Docker Compose



Production

Kubernetes



↓

Helm

↓

ArgoCD

CI/CD

GitHub Actions

Pipeline

Lint



↓

Tests

↓

Security Scan

↓

Docker Build

↓

Push GHCR

↓

Helm Lint

↓

Deploy Dev

↓

Integration Tests

↓

Promote

Security

Add

RBAC

OPA Gatekeeper

Kyverno

Network Policies

Pod Security Standards

Image Scanning

Cosign signing

SBOM generation

Milestone Plan (Recommended)

Phase 0 – Architecture & Foundation (Week 1)

- Define domain model and bounded contexts (Departments, Workflows, Tasks, Approvals, Tools).
- Set up monorepo structure and coding standards.
- Create Docker Compose development environment (PostgreSQL, Redis, NATS, Temporal, Qdrant, MinIO, Keycloak, Vault).
- Bootstrap FastAPI backend and Next.js frontend.
- Establish CI (linting, formatting, unit tests).

Phase 1 – Core Platform (Weeks 2–3)

- Authentication with Keycloak.
- Organization, user, and role management.
- Workflow engine with Temporal.
- Event bus using NATS JetStream.
- Planner agent using LangGraph.
- LiteLLM gateway and Ollama integration.
- Centralized configuration and secrets.

Phase 2 – Agent Framework (Weeks 4–5)

- Generic agent runtime.
- Memory layer (Redis + Qdrant + PostgreSQL).
- Tool abstraction via Composio.
- Approval framework.
- Audit logging.
- Agent telemetry with OpenTelemetry and Phoenix.

Phase 3 – Enterprise Departments (Weeks 6–8)

Implement the first four production-ready departments:

1. **Support** – Email/ticket intake, classification, customer communication.
2. **Engineering** – GitHub, code review, issue creation, documentation.
3. **Platform Engineering** – Kubernetes, Helm, Argo CD deployment operations.
4. **SRE** – Incident response, metrics analysis, log analysis, rollback recommendations.

Complete at least two end-to-end workflows (for example, production incident response and feature deployment).

Phase 4 – Production Infrastructure (Weeks 9–10)

- Package every service with Helm.
- Deploy to Kubernetes.
- Configure Argo CD for GitOps.

- Add Prometheus, Grafana, Loki, Jaeger, and Phoenix.
- Enable autoscaling and readiness/liveness probes.

Phase 5 – Enterprise Hardening (Weeks 11–12)

- Multi-tenancy.
 - Rate limiting and quotas.
 - Human approval policies.
 - Retry and compensation logic.
 - Comprehensive testing (unit, integration, end-to-end, load).
 - Security scanning and supply-chain protections.
-

The "wow factor"

To make this project memorable, I'd make **every action traceable**. When someone clicks on a workflow in the UI, they should be able to see:

- The planner's reasoning and task decomposition.
- Which agents participated.
- Which tools were called through Composio.
- The event timeline from NATS.
- The Temporal workflow graph.
- OpenTelemetry traces.
- Phoenix prompt and token metrics.
- Human approvals and decisions.
- Final outcome and audit log.

Very few open-source AI projects provide this level of visibility. It demonstrates that you're not just building agents—you understand how to build **observable, governable, production-grade AI systems**.

One architectural recommendation

Although the end goal is a microservice platform, **start with a modular monolith**:

- One FastAPI application containing all business modules (Planner, Support, Engineering, Platform, SRE).
- Separate modules/packages with clear interfaces and domain boundaries.
- NATS, Temporal, PostgreSQL, Redis, and the other infrastructure components running from day one.

Once the architecture stabilizes, split modules into independent services without rewriting core logic. This approach dramatically reduces early complexity while preserving a clean

path to microservices and is how many successful cloud-native platforms evolve.

If you follow this roadmap, you'll end up with something that resembles an internal AI platform at a company like Atlassian, GitLab, or HashiCorp—built almost entirely on open-source technologies, with only the language model provider being swappable between local models (Ollama) and hosted APIs through LiteLLM.